

in a system that someone might care about later. Someone logs in. A request hits an endpoint. A query runs against a database. A pod crashes at 2:14 a.m. for reasons you will spend the next hour trying to figure out. A payment fails.

Logs, metrics, and traces are three different ways to record those events. Logs keep each one as their own entry. Metrics turn events into numbers (how many, how often, how fast). Traces gather up the events that belong to one request, no matter how many services that request touched, and present them as a single story. Three angles on the same underlying material.

What a log is

A log is a written entry for one specific thing that happened, with a timestamp and whatever extra context the engineer who wrote the code included. Think of a security camera with the time stamped on every clip. Rewind to any moment and see exactly what was on the screen.

```
127.0.0.1 alice [06/May/2024:11:26:42]
GET /checkout 200 3477
```

Logs are painful to parse. Painful to search. Painful to do almost anything with once you have a few hundred million of them. Most teams now emit logs as structured JSON, which is the same event in a format a machine can handle:

```
{
  "user": "alice",
  "method": "GET",
  "path": "/checkout",
  "status": 200,
  "time": "2024-05-06T11:26:42Z"
}
```

Logs come in different varieties depending on the job (error logs for things going wrong, audit logs for who-did-what-and-when, access logs for everything hitting the front door, debug logs that nobody enables in production until something is already on fire). The thread connecting them is depth. When something specific breaks at a specific moment, the log is

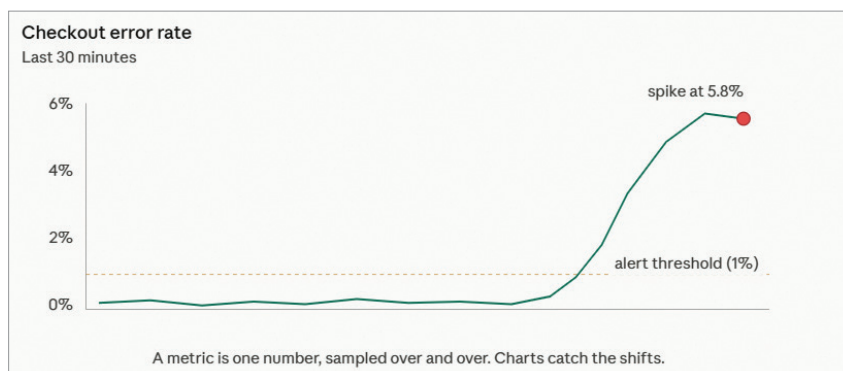


Figure 2: Metric chart

where the why is hiding.

What a metric is

A metric is a number, sampled again and again. The car speedometer is the cleanest analogy. It is not recording every mile you ever drove. It shows you how fast you are going right now, and a chart afterwards can show you what your speed did across the whole drive.

Metrics in a real system look like:

- CPU sitting at 67 percent
- 12,400 requests per minute hitting the checkout service
- Error rate of 0.4 percent over the last five minutes
- Average response time around 240 milliseconds

The reason teams reach for metrics first is cost. Storing one number per

minute is almost free. Querying months of those numbers is quick. You can glance at a chart and notice something shifted.

There is a popular short list called the ‘four golden signals’ (Google’s SRE book put it on the map and it stuck). The four are latency, traffic, errors, and saturation. Latency for how long requests take, traffic for how many are coming in, errors for how many are failing, and saturation for how full things are running. If these four look healthy, the service usually is. Not always. But usually.

What a trace is

A trace follows one request from end to end. The parcel-delivery analogy is the one I keep coming back to. A package gets a tracking number when it leaves

A quick comparison of the three pillars of observability

Pillar	What it answers	What it looks like	Where it shines	Where it falls short
Logs	What happened at a specific moment	A timestamped record of one event	Deep detail for a single moment	Volume grows fast, storage gets expensive
Metrics	Is something off, and how widespread	A number measured repeatedly over time	Cheap, fast, easy to chart	Tells you what, not why
Traces	Where in a request did time go	A journey of one request across services	Pinpointing slow steps in distributed systems	Needs every service instrumented